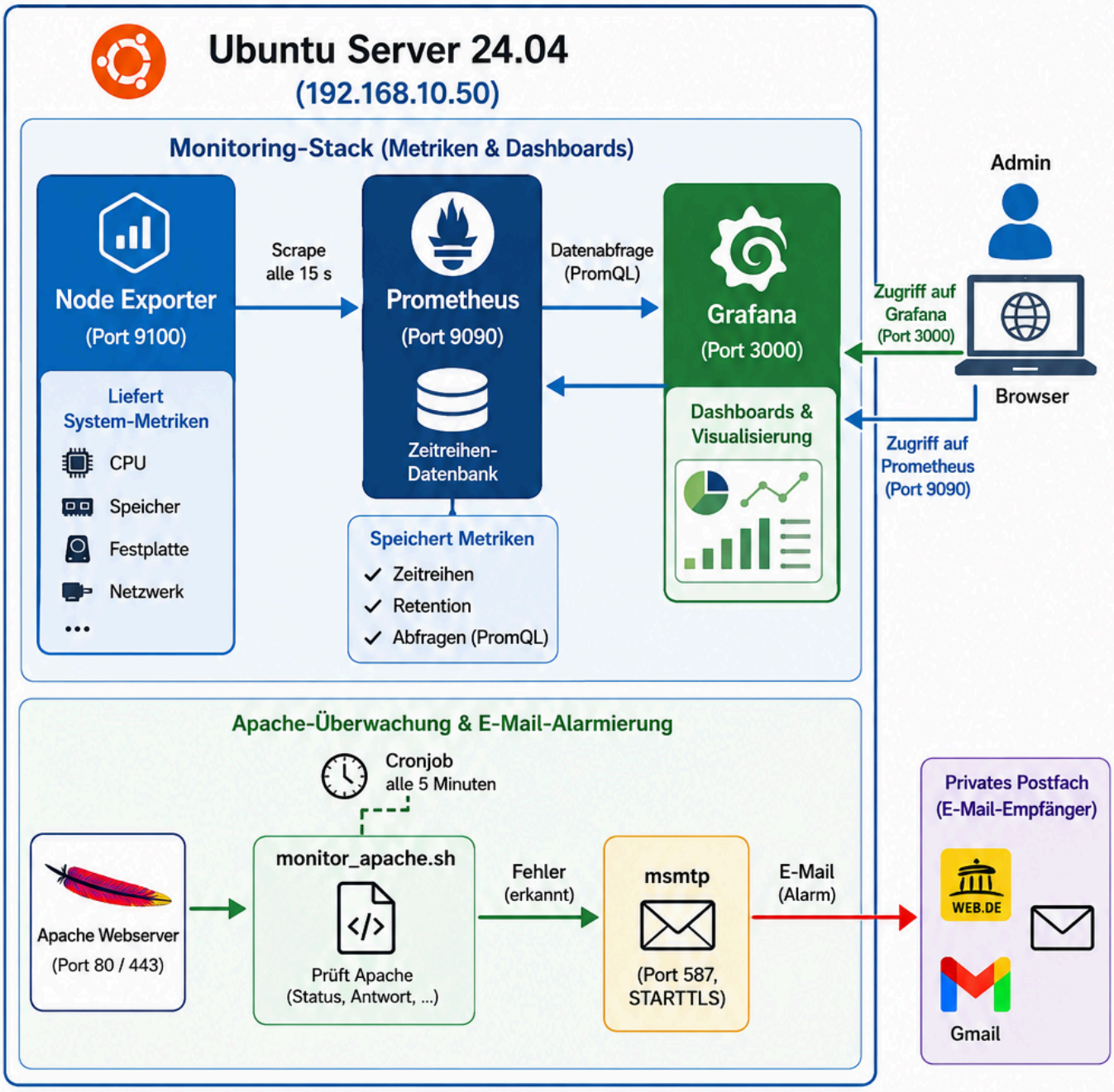
Schritt-für-Schritt: Monitoring mit Prometheus, Grafana und E-Mail-Alarm

LF10b · LS04 – Monitoring und Alerting einrichten · Anleitung · IT-3B · Schuljahr 2025/2026

🎯 **Ziel dieser Anleitung**

Sie statten Ihren Server mit einem professionellen Monitoring aus: Der **Node Exporter** liefert die Messwerte, **Prometheus** sammelt sie ein, **Grafana** zeigt sie als Dashboard – und über **msmtp** verschickt der Server Alarm-E-Mails. Voraussetzung: laufender Server aus LS01/LS02.

Monitoring-Stack – Gesamtüberblick



Monitoring-Stack

- Node Exporter (9100)
  Liefert System-Metriken
- Prometheus (9090)
  Scrape alle 15 s, speichert Zeitreihen, Abfragen (PromQL)
- Grafana (3000)
  Dashboards & Visualisierung

Zugriff (Admin)

- Browser → Grafana (3000)
  Dashboard-Zugriff
- Browser → Prometheus (9090)
  Abfragen & Status

Alarmierungs-Flow

1. Cron: alle 5 Minuten
2. monitor_apache.sh prüft Apache
3. Fehler erkannt
4. msmtp (Port 587, STARTTLS)
5. E-Mail an privates Postfach (web.de / Gmail)

Alle Komponenten laufen auf Ubuntu Server 24.04 (192.168.10.50).

Abbildung 1: Gesamtüberblick – diese Dienste richten Sie ein und so hängen sie zusammen (Node Exporter → Prometheus → Grafana; Alarmweg über msmtp)

Teil A – Vorbereitung

Schritt 1 | Snapshot und IP

Vorbereitung

```
sudo shutdown now      # dann in VirtualBox: Sicherungspunkt "vor_LS04_Monitoring"
# VM wieder starten, anmelden, IP pruefen:
ip a
```

Meine Server-IP heute	

Teil B – Node Exporter (die Messstation)

Schritt 2 | Node Exporter installieren und prüfen

Installation aus den Ubuntu-Paketquellen

```
sudo apt update
sudo apt install prometheus-node-exporter -y
sudo systemctl enable --now prometheus-node-exporter
systemctl status prometheus-node-exporter      # Active: active (running)?
```

Liefert er Messwerte? (Port 9100)

```
curl -s http://localhost:9100/metrics | head -20
```

Die Ausgabe sieht ungefähr so aus (Ihre Zahlenwerte unterscheiden sich):

Beispielausgabe von curl … /metrics | head -20

```
# HELP node_cpu_seconds_total Seconds the CPUs spent in each mode.
# TYPE node_cpu_seconds_total counter
node_cpu_seconds_total{cpu="0",mode="idle"} 12345.67
node_cpu_seconds_total{cpu="0",mode="system"} 231.45
node_cpu_seconds_total{cpu="0",mode="user"} 512.88
# HELP node_memory_MemAvailable_bytes Memory information field MemAvailable_bytes.
# TYPE node_memory_MemAvailable_bytes gauge
node_memory_MemAvailable_bytes 1.7423616e+09
# HELP node_filesystem_avail_bytes Filesystem space available to non-root users in bytes.
# TYPE node_filesystem_avail_bytes gauge
node_filesystem_avail_bytes{device="/dev/sda2",mountpoint="/"} 3.9123456e+10
```

So lesen Sie das Format: Zeilen mit `#` sind Kommentare – `# HELP` beschreibt die Metrik, `# TYPE` nennt ihren Typ (`counter` = zählt nur hoch, `gauge` = aktueller Momentwert). Die übrigen Zeilen sind die eigentlichen Messwerte im Schema `metrikname{label="wert"} zahl` – z. B. zeigt `node_memory_MemAvailable_bytes` den gerade verfügbaren Arbeitsspeicher in Byte (hier ca. 1,74 GB). Sehen Sie solche Zeilen, liefert der Node Exporter korrekt die Metriken Ihres Servers.

> 💡 **Kein UFW-Port nötig**
> Der Node Exporter (9100) wird nur **intern** von Prometheus abgefragt – Sie geben dafür **keinen** Firewall-Port frei. Das hält die Angriffsfläche klein.

Schritt 3 | Prometheus installieren

Installation und Funktionstest (Port 9090)

```
sudo apt install prometheus -y
sudo systemctl enable --now prometheus
curl -s http://localhost:9090/-/healthy        # Antwort: Prometheus Server is Healthy.
```

Schritt 4 | Scrape-Konfiguration kontrollieren

Konfiguration ansehen

```
sudo nano /etc/prometheus/prometheus.yml
```

Prüfen Sie, dass unter `scrape_configs:` ein Job auf den Node Exporter zeigt (beim Ubuntu-Paket bereits voreingestellt):

/etc/prometheus/prometheus.yml (Auszug – muss so ähnlich vorhanden sein)

```
scrape_configs:
  - job_name: node
    static_configs:
      - targets: ['localhost:9100']
```

Falls Sie etwas geändert haben: speichern und `sudo systemctl restart prometheus`.

Schritt 5 | Prometheus-Web-UI vom Host prüfen

Port 9090 freigeben

```
sudo ufw allow 9090/tcp
sudo ufw status
```

1. Host-Browser: `http://[Server-IP]:9090/targets`
2. Beide Targets müssen **State = UP** zeigen (prometheus selbst und node).

📸 **Screenshot 1: Targets-Seite mit State „UP"**	☐ erledigt

Teil C – Grafana (das Dashboard)

Schritt 6 │ Grafana-Paketquelle hinzufügen und installieren

Grafana liegt nicht in den Ubuntu-Quellen, sondern im Hersteller-Repository:

Repository einrichten (jede Zeile einzeln ausführen)

```
sudo apt install -y apt-transport-https software-properties-common wget
sudo mkdir -p /etc/apt/keyrings
wget -q -O - https://apt.grafana.com/gpg.key | sudo gpg --dearmor -o /etc/apt/keyrings/grafana.g
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable main" | sudo
```

Grafana installieren und starten (Port 3000)

```
sudo apt update
sudo apt install grafana -y
sudo systemctl enable --now grafana-server
systemctl status grafana-server                # Active: active (running)?
sudo ufw allow 3000/tcp
```

📖 **Was installieren Sie hier – und warum?**

- **apt-transport-https, software-properties-common, wget** – Hilfswerkzeuge, damit APT Pakete über eine verschlüsselte HTTPS-Verbindung laden kann und Sie den Schlüssel herunterladen können.
- **GPG-Schlüssel** (`grafana.gpg`) – der öffentliche Signaturschlüssel von Grafana. Damit prüft APT, ob die Pakete wirklich vom Hersteller stammen und unterwegs nicht verändert wurden.
- **Repository-Eintrag** (`grafana.list`) – trägt die Paketquelle `apt.grafana.com` ein. Das ist nötig, weil **Grafana nicht in den offiziellen Ubuntu-Paketquellen enthalten ist**; über das Hersteller-Repo bekommen Sie zudem immer die aktuelle, gepflegte Version.
- **grafana** – das eigentliche Programm: der Grafana-Server, ein Webdienst, der ab jetzt als systemd-Dienst `grafana-server` läuft und die Weboberfläche auf **Port 3000** bereitstellt.
- **ufw allow 3000/tcp** – gibt diesen Port frei, damit Sie das Dashboard vom Host-Browser aus erreichen. (Anders als der Node Exporter muss Grafana von außen erreichbar sein.)

Schritt 7 │ Erste Anmeldung und Passwort ändern

1. Host-Browser: `http://[Server-IP]:3000`
2. Anmelden mit `admin` / `admin`.
3. Grafana verlangt sofort ein **neues Passwort** – vergeben und **dokumentieren**:

Grafana-URL	

Grafana-Benutzer / neues Passwort	

Schritt 8 │ Prometheus als Datenquelle verbinden

1. Menü (☰) → **Connections** → **Data sources** → **Add data source** → **Prometheus**.
2. Bei **Prometheus server URL** eintragen: `http://localhost:9090`
3. Unten **Save & test** → grüne Meldung „Successfully queried …".

> 📖 **Warum localhost?**
> Grafana und Prometheus laufen auf **demselben** Server – die Verbindung bleibt über `localhost` auch dann stabil, wenn sich die externe IP per DHCP ändert.

Schritt 9 │ Fertiges Dashboard importieren

1. Menü (☰) → **Dashboards** → **New** → **Import**.
2. Dashboard-ID `1860` eingeben („Node Exporter Full") → **Load**.
3. Unten als Datenquelle Ihren **Prometheus** wählen → **Import**.
4. Das Dashboard zeigt CPU, RAM, Platte und Netzwerk Ihres Servers live.

📸 **Screenshot 2: Dashboard mit Live-Werten (CPU/RAM/Disk sichtbar)**	☐ erledigt

Schritt 9b │ Dienst-Status ins Dashboard aufnehmen

Das Dashboard zeigt bisher nur CPU/RAM/Platte. Jetzt holen wir die Informationen, die wir vorher nur über `systemctl status …` auf der Konsole gesehen haben (läuft **Apache2**? läuft **Prometheus**?), **direkt ins Grafana-Dashboard**.

1. Den passenden Sensor im Node Exporter einschalten. Der Dienst-Status wird vom *systemd-Collector* geliefert, der standardmäßig aus ist. Wir aktivieren ihn:

systemd-Collector aktivieren

```
sudo nano /etc/default/prometheus-node-exporter
```

Tragen Sie in die Zeile `ARGS=""` die folgenden Optionen ein (begrenzt auf unsere vier Dienste, damit das Dashboard übersichtlich bleibt):

/etc/default/prometheus-node-exporter

```
ARGS="--collector.systemd --collector.systemd.unit-include=(apache2|prometheus|prometheus-node-
```

Node Exporter neu starten und Sensor prüfen

```
sudo systemctl restart prometheus-node-exporter
curl -s http://localhost:9100/metrics | grep node_systemd_unit_state | grep 'state="active"'
```

Sie sehen jetzt Zeilen wie diese (Wert `1` = Dienst läuft, `0` = Dienst aus):

Beispielausgabe

```
node_systemd_unit_state{name="apache2.service",state="active",type="forking"} 1
node_systemd_unit_state{name="prometheus.service",state="active",type="simple"} 1
node_systemd_unit_state{name="grafana-server.service",state="active",type="simple"} 1
node_systemd_unit_state{name="prometheus-node-exporter.service",state="active",type="simple"} 1
```

2. Eine Status-Anzeige in Grafana anlegen. Wir bauen ein eigenes kleines Panel, das alle Dienste auf einen Blick zeigt:

1. Menü (☰) → **Dashboards** → **New** → **New dashboard** → **Add visualization** → als Datenquelle **Prometheus** wählen.
2. Oben rechts den Visualisierungstyp von „Time series" auf **„Stat"** umstellen.
3. Im Abfrage-Editor unten von *Builder* auf **Code** umschalten und genau diese Abfrage eintragen:

   Prometheus-Abfrage (eine Kachel je Dienst)

   ```
   node_systemd_unit_state{state="active", name=~"apache2.service|prometheus.service|prometheus
   ```

4. Rechts unter **Options** → **Legend** auf „Custom" stellen und `{{name}}` eintragen – so steht der Dienstname auf jeder Kachel.
5. Rechts den Bereich **Value mappings** öffnen → **Add value mappings**:
   - Value `1` → Anzeigetext **läuft**, Farbe **grün**.
   - Value `0` → Anzeigetext **DOWN**, Farbe **rot**.
6. Oben rechts auf **Apply**, dann das Dashboard mit dem Disketten-Symbol als **„NetConsult – Dienst-Status" speichern**.

> ✅ **Test: Status wird live aktualisiert**
> Stoppen Sie testweise Apache (`sudo systemctl stop apache2`) und aktualisieren Sie das Panel – die Apache-Kachel springt nach wenigen Sekunden auf **DOWN/rot**. Mit `sudo systemctl start apache2` wird sie wieder **grün**. So sehen Sie den Ausfall sofort im Dashboard – zusätzlich zur Alarm-E-Mail aus Teil D.

> 💡 **Hinweis: Skript-Lauf vs. Logzeilen**
> Das Skript `monitor_apache.sh` ist kein Dauerdienst, sondern läuft per Cron – es taucht daher nicht als „active"-Kachel auf; seine Arbeit sehen Sie in `/var/log/netconsult/monitor.log`. Möchte man die *Textzeilen* der Logdateien (z. B. `monitor.log` oder das msmtp-Log `/var/log/msmtp`) ebenfalls in Grafana anzeigen, ergänzt man später **Loki + Promtail** als Log-Datenquelle – das ist eine sinnvolle Erweiterung, aber nicht Teil dieser Grundeinrichtung.

📸 **Screenshot 3: Status-Panel mit allen Diensten „läuft" (grün)**	☐ erledigt

Teil D – E-Mail-Alarm mit msmtp

Schritt 10 │ msmtp installieren und konfigurieren

Installation

```
sudo apt install msmtp msmtp-mta mailutils -y
sudo nano /etc/msmtprc
```

💡 **Blaues Dialogfenster „AppArmor-Unterstützung aktivieren?"**
Während der Installation kann ein blaues Konfigurationsfenster erscheinen, das fragt, ob die **AppArmor-Unterstützung** aktiviert werden soll. AppArmor ist ein Sicherheitsmodul von Ubuntu, das einem Programm nur den Zugriff auf die Dateien und Ressourcen erlaubt, die es wirklich braucht – das schränkt den möglichen Schaden ein, falls das Programm angegriffen wird.

So bedienen Sie das Fenster:

- Wählen Sie **<Ja>** (empfohlen) – das AppArmor-Profil erlaubt die Konfigurationsdatei `/etc/msmtprc` und als Logdatei genau `/var/log/msmtp` (ohne Endung `.log`!). Deshalb steht in den Beispielen unten `logfile /var/log/msmtp` – ein anderer Logpfad würde von AppArmor blockiert.
- Bedienung mit der Tastatur: mit den **Pfeiltasten** oder **Tab** zwischen `<Ja>` und `<Nein>` wechseln, mit **Enter** bestätigen (die Maus funktioniert in diesem Textdialog nicht).

Es gibt keinen Schul-SMTP-Server – **jede/r nutzt das eigene private E-Mail-Konto** als Versand-Server. Damit ein Programm wie msmtp über Ihr Konto senden darf, müssen Sie den Zugriff durch externe Programme im Webmail-Konto **erst freischalten**. Wählen Sie unten Ihren Anbieter.

Variante A – Web.de
1. Im Web.de-Konto freischalten (sonst lehnt der Server die Anmeldung ab):

- Bei `web.de` im Browser anmelden.
- Links im Menü auf **Einstellungen** klicken (Zahnrad-Symbol).
- Den Punkt **„POP3 / IMAP Abruf"** öffnen.
- Den Haken bei **„POP3 und IMAP Zugriff erlauben"** setzen und **speichern**.

2. Zugangsdaten: Benutzer = Ihre volle Adresse (`name@web.de`), Passwort = Ihr normales Web.de-Passwort.

/etc/msmtprc – Beispiel Web.de

```
defaults
auth           on
tls            on
tls_trust_file /etc/ssl/certs/ca-certificates.crt
logfile        /var/log/msmtp
```

```
account        webde
host           smtp.web.de
port           587
from           max.mustermann@web.de
user           max.mustermann@web.de
password       IHR-WEB.DE-PASSWORT

account default : webde
```

Variante B – Gmail / Googlemail

Google lässt das normale Konto-Passwort für externe Programme **nicht** zu. Sie brauchen ein **App-Passwort**, und dafür muss zuerst die Zwei-Faktor-Anmeldung aktiv sein:

- Google-Konto öffnen: `myaccount.google.com` → **Sicherheit**.
- **„Bestätigung in zwei Schritten"** aktivieren (falls noch nicht geschehen).
- Danach erscheint dort der Punkt **„App-Passwörter"** – öffnen.
- Einen Namen vergeben (z. B. `msmtp-Server`) → Google erzeugt ein **16-stelliges Passwort**.
- Dieses 16-stellige Passwort (ohne Leerzeichen) als `password` eintragen – *nicht* Ihr normales Google-Passwort.

`/etc/msmtprc – Beispiel Gmail`

```
defaults
auth           on
tls            on
tls_trust_file /etc/ssl/certs/ca-certificates.crt
logfile        /var/log/msmtp

account        gmail
host           smtp.gmail.com
port           587
from           max.mustermann@gmail.com
user           max.mustermann@gmail.com
password       APP-PASSWORT-16-ZEICHEN

account default : gmail
```

`Rechte einschränken (Pflicht – sonst kann jeder Benutzer das Passwort lesen)`

```
sudo chmod 600 /etc/msmtprc
```

⚠️ Passwort in Konfigdatei

Die Datei enthält Ihr Klartext-Passwort bzw. App-Passwort – deshalb `chmod 600` (nur root darf lesen). Folge: msmtp funktioniert nur noch mit `sudo` bzw. als root (z. B. in der Root-Crontab) – genau so ist es gewollt. Verwenden Sie bei Gmail **immer ein App-Passwort** (jederzeit widerrufbar), nie Ihr Haupt-Passwort. App-Passwort/Konto nach dem Kurs ggf. wieder entfernen.

Schritt 11 | Testmail senden

Schicken Sie die Testmail am besten an Ihre **eigene Adresse**, dann können Sie sofort prüfen, ob sie ankommt. Geben Sie die Mail **direkt an msmtp** – dabei müssen Betreff und Text Teil der Nachricht sein:

Testmail mit Betreff + Inhalt und Log-Kontrolle

```
echo -e "Subject: Test vom NetConsult-Monitoring\n\nDies ist eine automatische Test-E-Mail aus
sudo tail -5 /var/log/msmtp                    # exitcode=EX_OK = erfolgreich versendet
```

💡 **Warum `echo -e` und die `Subject:`-Zeile?**
msmtp verschickt den Text *genau so*, wie er hereinkommt – es ergänzt keinen Betreff. Deshalb muss die Nachricht die Kopfzeile `Subject: …` selbst enthalten, danach eine **Leerzeile**, und erst dann den eigentlichen Text. `echo -e` sorgt dafür, dass die `\n` als echte Zeilenumbrüche gesendet werden (`\n\n` = die nötige Leerzeile zwischen Betreff und Text). Ohne diese Trennung kommt die Mail ohne Betreff oder ganz ohne Inhalt an.

Testmail angekommen (mit Betreff + Text)?	☐ Ja ☐ Nein
Relevante Logzeile (/var/log/msmtp)	

Schritt 11b | E-Mail-Alarm ins Monitoring-Skript einbauen und automatisch ausführen

Bisher kann der Server zwar E-Mails senden (Schritt 11) – aber noch verschickt *niemand automatisch* eine Warnung, wenn Apache ausfällt. Das übernimmt jetzt ein Monitoring-Skript, das **per Cronjob alle 5 Minuten** läuft. Wir erweitern das Skript aus LS03 um eine `send_alert()`-Funktion und speichern es als neue Version `monitor_apache_v2.sh`.

Skript anlegen

```
sudo mkdir -p /var/log/netconsult
sudo nano /opt/scripts/monitor_apache_v2.sh
```

/opt/scripts/monitor_apache_v2.sh

```
#!/bin/bash
EMPFAENGER="ihre-eigene-adresse@web.de"     # dieselbe Adresse wie beim Test
LOGDATEI="/var/log/netconsult/monitor.log"
HOST=$(hostname)

log() { echo "$(date '+%Y-%m-%d %H:%M:%S') - $1" >> "$LOGDATEI"; }

send_alert() {
    echo -e "Subject: [ALERT] Apache auf $HOST - $1\n\nServer:    $HOST\nZeitpunkt: $(date '+%Y-
    log "E-Mail-Alarm gesendet an $EMPFAENGER ($1)"
```

```
}

if systemctl is-active --quiet apache2; then
    log "Apache OK"
else
    log "WARNUNG: Apache nicht aktiv - Neustart wird versucht"
    systemctl restart apache2
    if systemctl is-active --quiet apache2; then
        log "Neustart erfolgreich"
        send_alert "Neustart erfolgreich"
    else
        log "FEHLER: Neustart fehlgeschlagen"
        send_alert "Neustart FEHLGESCHLAGEN - manueller Eingriff noetig"
        exit 1
    fi
fi
```

Skript ausführbar machen

```
sudo chmod +x /opt/scripts/monitor_apache_v2.sh
```

Jetzt den Cronjob einrichten – damit das Skript automatisch (ohne dass jemand es startet) alle 5 Minuten läuft. Da es `systemctl restart` ausführt und nach `/var/log` schreibt, braucht es Root-Rechte → wir nutzen die **Root-Crontab**:

Cronjob anlegen / aktualisieren

```
sudo crontab -e
```

Tragen Sie folgende Zeile ein. **Wichtig:** Falls aus LS03 noch eine Zeile auf das alte `monitor_apache.sh` zeigt, **ersetzen** Sie diese – sonst läuft die alte Version *ohne* E-Mail-Alarm weiter:

Zeile in der Root-Crontab

```
*/5 * * * * /opt/scripts/monitor_apache_v2.sh >> /var/log/netconsult/cron.log 2>&1
```

Eintrag kontrollieren

```
sudo crontab -l
```

📖 **Wann und wie wird das Skript ausgeführt?**

- **Automatisch, alle 5 Minuten** durch den cron-Dienst – Sie müssen es nicht von Hand starten.
- Bei jedem Lauf prüft es, ob Apache aktiv ist. **Läuft Apache → nur Log-Eintrag „Apache OK", keine Mail.**
- **Ist Apache down → Neustart-Versuch → E-Mail** mit dem Ergebnis („Neustart erfolgreich" oder „FEHLGESCHLAGEN"). So erhalten Sie spätestens 5 Minuten nach einem Ausfall automatisch eine Warnung.

- Weil cron als **root** läuft, findet es die geschützte Datei `/etc/msmtprc` und darf Apache neu starten.

✅ Test: automatischer Alarm bei Ausfall

Apache stoppen und – um nicht 5 Minuten zu warten – das Skript einmal von Hand auslösen:

Ausfall simulieren

```
sudo systemctl stop apache2
sudo /opt/scripts/monitor_apache_v2.sh        # einmaliger Test ohne auf den Cron zu warte
sudo tail -5 /var/log/netconsult/monitor.log
```

Erwartung: Sie erhalten eine **Alarm-E-Mail**, und im Log steht „Neustart erfolgreich". Apache läuft danach wieder. (Für den Fehlerfall „FEHLGESCHLAGEN" wie in der Lernsituation den Dienst zusätzlich mit `sudo systemctl mask apache2` blockieren – danach mit `unmask` wieder freigeben.)

Crontab-Zeile dokumentiert	
Alarm-E-Mail bei Ausfall erhalten?	☐ Ja ☐ Nein

Schritt 12 | Abschluss-Dokumentation

Node Exporter erreichbar unter (intern)	
Prometheus-Web-UI (URL)	
Grafana-Dashboard (URL + Name)	
Freigegebene Firewall-Ports (gesamt)	
Alarmweg (Programm + Empfängeradresse)	

✅ Fehlersuche

- Target „DOWN" in Prometheus? → `systemctl status prometheus-node-exporter` und `prometheus.yml` prüfen.
- Grafana nicht erreichbar? → `systemctl status grafana-server`, `sudo ufw status` (Port 3000?).
- Keine Testmail? → `sudo tail -20 /var/log/msmtp` – dort steht die genaue Fehlermeldung (z. B. falsches Passwort, Port geblockt).

💡 **Geschafft!**
Ihr Server überwacht sich jetzt selbst: Messwerte → Prometheus → Grafana-Dashboard, plus E-Mail-Weg für Alarme. In den Aufgaben der Lernsituation testen (A5) und dokumentieren (A6) Sie darauf aufbauend das Gesamtsystem.